

Yazılım Mühendisliğine Giriş ve Dönem Projesi

Erciyes Üniversitesi
Bilgisayar Mühendisliği Bölümü

2026–2027 Güz

Bu haftanın hedefleri

Bu dersin sonunda şunları yapabileceksiniz:

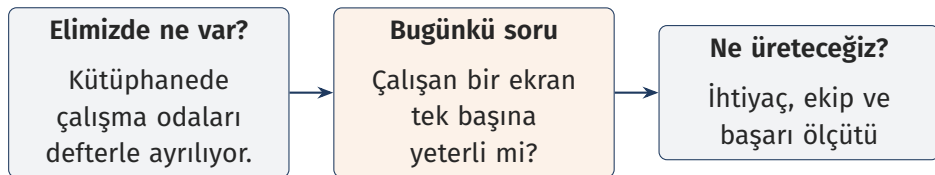
- Programlama ile yazılım mühendisliği arasındaki farkı güncel vakalarla açıklayabileceksiniz.
- Bir yazılım kesintisinin gerçek maliyetini kalemlere ayırarak hesaplayabileceksiniz.
- Proje sahibi, müşteri ve kullanıcı rollerini bir sistem üzerinde ayırt edebileceksiniz.
- Yazılım geliştirmenin yaşam döngüsü adımlarını sıralayıp bu dersin hangi haftasında ele alındığını bileceksiniz.
- Dönem projesinin kurallarını uygulayıp grubunuzu ve müşteri adaylarınızı belirleyebileceksiniz.

- Açılış, hedefler, bugünün akışı
- Bölüm 1: Yazılım mühendisliği neden zor?
- Uygulamalı örnek ve alıştırma: kesintinin maliyeti
- Bölüm 2: Roller, mesleki sorumluluk, yaşam döngüsü
- Uygulamalı örnek ve alıştırma: kapsamı daraltmak
- Bölüm 3: Dönem projesi kuralları ve grup kurma
- Bu hafta projenizde, kaynaklar, kapanış

Gruplar oluştuktan sonra: **her grup bir slayt, bir dakika.**

- Haftanın sorusunu yanıtlayın: yapılan iş, müşteri görüşmesi, bir engel.
- Slaytı dersten önce grupta belirtilen forma yükleyin.
- Konuşmacıyı her hafta değiştirin; herkes dönem içinde konuşsun.
- Süre bitince söz sonraki gruba geçer.

Çalışan koddan çözülmüş probleme



Önce düşünelim

Dönem boyunca aynı örneği kullanacağız: kütüphane çalışma odası rezervasyonu. Rezervasyon düğmesi çalıştığı hâlde iki öğrenci aynı oda için kapıda karşılaşırsa projeyi başarılı sayar mısınız? Neden?

Ortak örnek: kütüphane çalışma odası rezervasyonu. Sayılar ve kurallar öğretim amaçlı varsayımlardır.

Bölüm 1: Yazılım mühendisliği neden zor?

Bu ders neyi öğretir?

- Programlamayı zaten biliyorsunuz. Bu ders programlama dersi değildir.
- Konu şu: *birden çok kişi, gerçek bir müşteri için, sınırlı sürede çalışan ve bakımı yapılabilir bir yazılımı nasıl üretir?*
- Bu sorunun teknik olduğu kadar örgütsel ve insani yanı da vardır.

Yazılım mühendisliği

Yazılımın belirlenmesi, tasarımı, geliştirilmesi, doğrulanması, teslimi ve bakımı için sistematik, ölçülebilir ve yinelenebilir yaklaşımların uygulanmasıdır.

Programlama sorusu

“Boş odayı bulup rezervasyonu nasıl kaydederim?”

Algoritma, veri yapısı ve doğru çalışan kod.

Mühendislik soruları

Bu kuralı kim onayladı?
İki istek çakışırsa ne olur?
Ekibin başka üyesi kodu değiştirebilir mi?

Birbirini tamamlayan işler

Programlama çözümü kurar. Yazılım mühendisliği; ihtiyaç, ekip, kalite, teslimat ve bakım kararlarını bu çözüme bağlar.

Emeği takvimden ayırın

Kişi-saat harcanan emeği, **hafta** geçen süreyi gösterir. Sekiz kişinin aynı projede bulunması, sekizinin de tam zamanlı çalıştığı anlamına gelmez.

Dönem projesi için varsayımsal hesap

$8 \text{ kişi} \times 12 \text{ hafta} \times \text{haftada } 5 \text{ saat} = \mathbf{480 \text{ kişi-saat.}}$

Bir tam zamanlı kişi-ayını 160 saat varsayarsak: $480/160 = 3 \text{ kişi-ay.}$

- Üç kişi-ay emek, projenin üç takvim ayında biteceği anlamına gelmez.
- Görüşme, inceleme, test ve düzeltme de bu emeğin içindedir.

Maliyet nereye gider?

Yaygın sanı

- Zamanın çoğu kod yazmaya gider.
- Yazılım bitince iş biter.
- Belge zaman kaybıdır.

Gözlenen durum

- Kodlama, toplam emeğin küçük bir dilimidir.
- Bakım, çoğu sistemde en büyük kalemdir.
- Belgesiz sistemin bakımı, yeniden yazmaktan pahalı olabilir.

- Bir sistemi anlamak, değiştirmekten uzun sürer. Okunabilirlik bu yüzden maliyet konusudur.
- “Sonra düzeltiriz” kararı bir borçtur ve faiziyle geri döner.

Yazılım çok çeşitlidir

- **Uygulamalar:** web siteleri, mobil uygulamalar
 - **Sistem yazılımı:** işletim sistemleri, derleyiciler
 - **Gömülü:** sürücüler, denetleyiciler
 - **Gerçek zamanlı:** trafik yönetimi, otonom araç
 - **Veri işleme:** faturalama, bordro
 - **Bilgi sistemleri:** veri tabanları, arşivler
 - **Bilimsel:** benzetim, hava tahmini
 - **Yapay zekâ:** model eğitimi, çıkarım servisleri
-
- Bir uçuş kontrol yazılımının kabul ölçütü ile bir kampanya sayfasının ölçütü aynı değildir.
 - Bu yüzden her projeye uyan tek bir süreç, tek bir dil, tek bir mimari yoktur.

- Yetkin bir geliştirici birçok yaklaşımı, yöntemi ve aracı *bilir*.
- Uсталık, bunların hangisinin bu projeye uygun olduğunu *seçmektir*.
- Seçim ölçütleri: belirsizlik düzeyi, hata bedeli, ekip deneyimi ve teslim baskısı.

Araç merkezli düşünme tuzağı

“Hangi çerçeveyi kullanalım?” sorusu, “müşteri hangi problemi çözmek istiyor?” sorusundan önce gelirse proje büyük olasılıkla yanlış şeyi üretir.

Vaka 1: CrowdStrike kesintisi (Temmuz 2024)

Ne oldu?

Bir güvenlik yazılımı için yayımlanan hatalı içerik güncellemesi, Windows çalıştıran milyonlarca kurumsal makinenin açılışta çökmesine yol açtı. Havayolları, hastaneler, bankalar ve yayıncılar saatlerce hizmet veremedi.

- Hata çekirdek düzeyinde çalışan bir bileşende oluştu; makineler kendiliğinden toparlanamadı.
- Düzeltme yayımlandıktan sonra da her makineye elle müdahale gerekti.
- Sorun yalnızca hatalı güncelleme değildi; güncellemenin sınanmadan ve kademeli yayımlanmadan dağıtılması da kesintiyi büyüttü.

Eksik olan

- Kademeli yayım: önce küçük bir gruba, sonra herkese.
- Girdi doğrulama: bozuk içerik dosyasına karşı savunma.
- Geri alma yolu: uzaktan eski sürüme dönüş.

Tek bir bileşenin arızalanarak tüm sistemi durdurmasına *tek hata noktası* denir. Hafta 9'da mimari düzeyde ele alacağız.

Bu derste nerede?

- Hafta 2: sürekli teslimat ve kademeli yayım.
- Hafta 10: sağlam programlama, girdi doğrulama.
- Hafta 11: test ve geri alma stratejisi.

Ne oldu?

Posta şubelerinin muhasebe sistemi Horizon, kasa açıklarını hatalı raporladı. Şube işletmecileri bu kayıtlara dayanarak suçlandı; yüzlerce kişi haksız biçimde mahkûm edildi. Süreç 2024'te geniş bir kamu soruşturmasına dönüştü.

- Yazılım hatası biliniyordu; ancak kurum “sistem hata yapmaz” varsayımını sürdürdü.
- Hatalı çıktı, tek ve tartışılmaz kanıt olarak kullanıldı.
- Sonuç yalnızca mali değil; insanların hayatı, itibarı ve özgürlüğü etkilendi.

- Bir yazılımın çıktısı, bir insan hakkında karar veriyorsa *itiraz yolu* tasarımın parçasıdır.
- Hata kayıtlarını görmezden gelmek teknik değil kurumsal bir seçimdir.
- Geliştiricinin sorumluluğu, bildiği hatayı yazılı olarak bildirmekle başlar.

Tartışma (5 dk)

- Yazılımın hatalı olduğunu bilen bir geliştirici, kurum susmayı seçtiğinde ne yapmalıdır?
- “Ben yalnızca kodu yazdım” savunması nerede biter?

Ne oldu?

Uçağın burnunu otomatik olarak aşağı iten MCAS yazılımı, tek bir hücum açısı sensöründen gelen veriye göre karar veriyordu. Sensör hatalı okuma yaptığı anda sistem tekrar tekrar devreye girdi. İki uçak düştü; dünyadaki tüm 737 MAX uçakları (yaklaşık 390 uçak) yaklaşık 20 ay boyunca uçuştan alındı.

- Yedekli sensör vardı ama yazılım yalnızca birini okuyordu.
- Pilotlar sistemin varlığından ve nasıl devre dışı bırakılacağından yeterince haberdar değildi.
- Belgeleme ve eğitim eksiklikleri de yazılım tasarımındaki sorunların bir parçasıydı.

Üç vakanın ortak dersi

Vaka	Kök neden nerede?	Bu derste hangi hafta?
CrowdStrike	Yayım süreci ve geri alma	2, 11
Horizon	Gereksinim, güvenilirlik, etik	4, 11, 12
737 MAX	Mimari yedeklilik, belgeleme	9, 12

- Üç vakada da sorun, tek bir kod hatasına indirgenemeyecek kadar genişti.
- Süreçteki eksikler, yanlış varsayımlar ve iletişim sorunları teknik kusurların etkisini büyüttü.
- Yazılım mühendisliği tam olarak bu üç alanı disiplin altına almaya çalışır.

Başarı ve başarısızlık kimin gözünden?

Teknik bakış

- Kod temiz mi?
- Testler geçiyor mu?
- Mimari zarif mi?
- Teknik olarak kusursuz ama kimsenin kullanmadığı sistem başarısızdır.
- Müşterinin işini çözen fakat bakımı zor olan bir sistem de uzun vadede sorun çıkarır. Kullanıcıya yararı ve teknik kaliteyi birlikte gözetin.

Müşteri ve yönetim bakışı

- İşimi çözüyor mu?
- Zamanında ve bütçede mi?
- Kullanıcılar kullanabiliyor mu?

Başarısızlığın üç klasik yüzü

Geç teslim, bütçe aşımı, beklenen işi yapmama. Genellikle üçü birlikte gelir.



- Kapsam, zaman ve maliyet birlikte değerlendirilir.
- Bu derste bir yarıyıl ve 7–10 kişiyle çalışırsınız; kapsamı buna göre seçin.
- Kaliteyi kıskarak zaman kazanmak, borç almaktır; borç dönem sonunda ödenir.

Görünürlük problemi

Yazılım geliştirmede ilerleme, dışarıdan bakan biri için ölçülemez. “Yüzde seksen bitti” cümlesi çoğu zaman doğrulanamaz.

- Görünmeyen ilerleme, geç fark edilen sapmaya yol açar.
- Sapma geç fark edilirse düzeltme maliyeti katlanır.
- Çözüm: ilerlemeyi *çalışan çıktı* ile göstermek.

- **Kısa döngüler:** iki haftada bir çalışan bir şey gösterin.
- **Erken müşteri teması:** haftada en az bir görüşme.
- **Görünür pano:** kim neyi yapıyor, herkes görsün.
- **En riskli parçayı öne alın:** bilinmeyeni sona bırakmayın.
- **Yazılı kararlar:** yalnızca sözlü kalan kararlar zamanla farklı hatırlanır.

☰ Bu hafta projenizde

Bu beş maddenin hepsi dönem projesinin zorunlu kuralı hâline getirilmiştir. Bunlar not için değil, projenizin bitmesi için vardır.

Uygulamalı örnek: bir kesintinin gerçek maliyeti

Senaryo

Bir çevrimiçi satış işletmesinin sitesi **2 saat** boyunca erişilemez oldu. Elimizdeki veriler:

Saatlik ortalama kâr	100.000 TL
Kesinti süresi	2 saat
Kesinti sırasında sipariş vermek isteyen müşteri sayısı	500 kişi
Bu müşterilerin kalıcı olarak kaybedilme oranı	%20
Bir müşterinin yıllık ortalama kâr katkısı	3.000 TL
Sözleşmedeki hizmet düzeyi cezası	50.000 TL
Olaya müdahale eden ekip	6 kişi × 8 saat
Mühendis saat maliyeti	500 TL

Soru: bu iki saatin işletmeye toplam maliyeti nedir?

Uygulamalı örnek: adım adım

▶ Adım 1 — Doğrudan gelir kaybı

$$2 \text{ saat} \times 100.000 = \mathbf{200.000 \text{ TL}}$$

▶ Adım 2 — Kalıcı müşteri kaybı

Kaybedilen müşteri: $500 \times 0,20 = 100$ kişi.

Yıllık kâr etkisi: $100 \times 3.000 = \mathbf{300.000 \text{ TL}}$

▶ Adım 3 — Sözleşme cezası

Sabit tutar: **50.000 TL**

▶ Adım 4 — Müdahale emeği

$$6 \times 8 \times 500 = \mathbf{24.000 \text{ TL}}$$

Uygulamalı örnek: sonuç ve yorum

Kalem	Tutar (TL)	Pay
Doğrudan gelir kaybı	200.000	%34
Kalıcı müşteri kaybı	300.000	%52
Sözleşme cezası	50.000	%9
Müdahale emeği	24.000	%4
Toplam	574.000	%100

- Doğrudan kayıp, toplamın yalnızca üçte biridir. Asıl bedel *güven kaybıdır*.
- Bu yüzden güvenilirlik bir “ekstra özellik” değil, doğrudan finansal bir gereksinimdir.
- İtibar kaybı ve yönetim dikkatinin başka işten çekilmesi bu tabloda hiç yok.

📅 Alıştırma (8 dk — ikili çalışın)

Bir üniversitenin öğrenci bilgi sistemi, **ders kayıt haftasının ilk günü** 4 saat boyunca çöktü. Aşağıdaki maliyet kalemlerini *parasal olan ve parasal olmayan* diye ikiye ayırın, sonra hangisinin daha ağır olduğunu tartışın.

- 40.000 öğrencinin kayıt yapamaması
- Çağrı merkezine gelen 3.000 çağrı
- Bilgi işlem ekibinin gece mesaisi
- Kayıt takviminin bir gün kayması
- Sosyal medyada kurum itibarı
- Kontenjan hatası nedeniyle açılan itirazlar

Çözüm

Parasal: gece mesaisi (doğrudan ücret), çağrı merkezi kapasite artışı, takvim kayması nedeniyle uzayan personel görevlendirmesi, itirazların idari işlem maliyeti.

Parasal olmayan ama daha ağır: 40.000 öğrencinin zaman kaybı ve stresi, kuruma güvenin azalması, yanlış kontenjan atamalarının akademik sonuçları, bir sonraki kayıt döneminde herkesin sisteme aynı anda yüklenmesi (davranış değişimi).

- Kamu ve eğitim sistemlerinde maliyet çoğu zaman *kuruma değil kullanıcıya* yüklenir.
- Bu, ölçülmediği için önemsiz sanılır. Gereksinim yazarken bunu açıkça ölçün.

Bölüm 2: Roller, mesleki sorumluluk ve yaşam döngüsü

Proje sahibi (client)

Projeyi sipariş eden, kapsamı onaylayan taraf. Sözleşmenin muhatabıdır.

Müşteri

Parayı ödeyen taraf. Faydanın karşılığını arar.

Kullanıcı

Sistemi fiilen kullanan kişi. Genelde kararlarda söz sahibi değildir.

- Üçü aynı kişi olabilir; çoğu gerçek projede değildir.
- Gereksinimleri kimden aldığınız, hangi rolle konuştuğunuza bağlıdır.

Örneklerle ayırım

Ürün	Proje sahibi	Müşteri	Kullanıcı
Netflix	Netflix ürün ekibi	Aboneliği ödeyen aile	Dizi izleyen sen
Üniversite bilgi sistemi	Öğrenci İşleri	Üniversite büt- çesi	Öğrenci, öğretim üyesi
Banka mobil uygulaması	Dijital kanallar bi- rimi	Banka	Hesap sahibi
Kampüs yemek siparişi	Kantin işletmecisi	Kantin işletmecisi	Öğrenci ve perso- nel
Hastane randevu sistemi	Sağlık müdürlüğü	Kamu bütçesi	Hasta ve hekim

Dönem projenizde onay formunu *proje sahibi* imzalar. Kullanıcıları ayrıca bulmanız gerekir.

- Yalnızca proje sahibiyle konuşursanız: mevzuata uygun ama kullanılamaz bir sistem çıkar.
- Yalnızca kullanıcıyla konuşursanız: sevilen ama onaylanmayan, bütçesiz bir sistem çıkar.
- Yalnızca müşteriyle konuşursanız: ucuz ama kimsenin işini çözmeyen bir sistem çıkar.

Tipik çatışma

Proje sahibi “her işlem kaydı 10 yıl saklanmalı” der. Kullanıcı “sayfa 8 saniyede açılıyor, kullanamıyorum” der. İkisi aynı gereksinimin iki yüzüdür ve tasarımda çözülür.

Alıştırma (7 dk — gruplarınızla)

Aşağıdaki üç sistem için proje sahibini, müşteriyi ve kullanıcıyı ayrı ayrı yazın. Sonra “bu sistemde en çok kimin sesi duyulmaz?” sorusunu yanıtlayın.

1. Bir belediyenin çöp toplama rotası optimizasyon yazılımı.
2. Bir okul için veli bilgilendirme uygulaması.
3. Bir fabrikanın üretim hattı arıza takip sistemi.

Alıştırma 2: çözüm

Sistem	Proje sahibi	Müşteri	Kullanıcı	Sesi duyulmayan
Çöp rotası	Temizlik İşleri Md.	Belediye büt- çesi	Şoför, ekip	Şoför ve mahalle sakini
Veli uygulaması Arıza takip	Okul yönetimi Üretim müdür- lüğü	Okul veya veli Fabrika	Veli, öğretmen Bakım teknis- yeni	Öğretmenin iş yükü Vardiya teknisyeni

- Üç örnekte de sesi duyulmayan taraf, sistemi *en çok kullanan* taraftır.
- Bu yüzden gereksinim toplarken sahaya gitmek, toplantı odasında kalmaktan değerlidir.

Kurumlar yazılımcıya neden güvenmek zorunda?

- **Yetkinlik:** ürettiğiniz yazılımın kalitesi doğrudan kurumun başarısını etkiler.
- **Gizlilik:** ticari sır, müşteri verisi ve kişisel veriye erişiminiz olur.
- **Yasal ortam:** telif, lisans ve veri koruma kuralları içinde çalışırsınız.
- **Kötüye kullanım:** yetkinizi aşan erişim, kurumu felç edebilir.

Mesleki sorumluluk

Teknik yetkinliğin yanında, verdiğiniz kararların başkaları üzerindeki sonuçlarını öngörme ve bunları açıkça bildirme yükümlülüğüdür.

- Test için üretim veri tabanının kopyasını almak, en yaygın veri ihlali biçimidir.
- Kişisel veri işleyen her sistem, Türkiye’de 6698 sayılı KVKK kapsamındadır.
- Bu, dönem projenizi de bağlar: gerçek kullanıcı verisi topluyorsanız kural aynıdır.

Dönem projesinde uygulanacak kural

Gerçek kişisel veri toplamayın. Kullanıcı testi için takma ad ve üretilmiş veri kullanın. Gerçek veri gerekiyorsa müşteriden yazılı izin alın ve Hafta 12’de ele alacağımız aydınlatma metnini hazırlayın.

- Yazılım mühendisliği iş birliğine dayanır; başkasının işine dayanmak normaldir.
- Anormal olan, dayandığınız kaynağı gizlemektir.
- Kural basit: kullandığınız her dış parçanın kaynağını ve lisansını yazın.

Doğru atıf

Kaynak: stackoverflow.com/a/123456 (CC BY-SA 4.0)

Değişiklik: hata yönetimi ve tip denetimi eklendi.

Lisanslar: üç yaygın aile

Aile	Ne verir?	Ne ister?
MIT, BSD	Neredeyse serbest kullanım	Lisans metnini ve telif notunu koru
Apache-2.0	Serbest kullanım, patent koruması	Değişiklikleri belirt, lisansı koru
GPL-3.0	Serbest kullanım	Türev çalışmayı aynı lisansla açık yayımla

- Kapalı kaynak bir ürüne GPL kod katmak, ürünü de açmak zorunda bırakabilir.
- Lisansı okumadan bağımlılık eklemek, hukuki bir karar vermektir.
- Hafta 12'de Türk mevzuatı ile birlikte ayrıntılı ele alacağız.

BZ 313 yapay zekâ ilkesi

Yapay zekâ araçları (ChatGPT, Claude, Copilot ve benzerleri) **kullanılabilir**. Üç koşul vardır: nerede kullandığınızı belirtin; ürettiği kodu anlamadan teslim etmeyin; doğruluğunu testle gösterin.

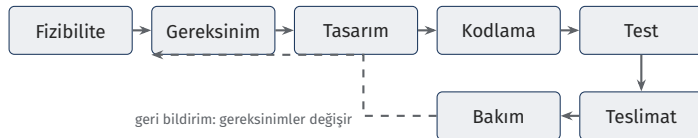
- İyi kullanım: kalıp kod, test iskeleti, bilmediğiniz bir hatanın açıklaması.
- Kötü kullanım: “fizibilite raporu yaz” deyip çıktıyı okumadan teslim etmek.
- Sorumluluk her durumda sizdedir. Araç, imzayı sizin adınıza atmaz.

Tartışma (6 dk)

- Bir yapay zekâ asistanının ürettiği kod, müşteride veri kaybına yol açtı. Sorumluluk kimdedir: aracı kullanan geliştiricide, ekip liderinde, aracı üretende?
- Kodun yapay zekâ ile üretildiğini müşteriye söylemek zorunda mısınız?
- Bir işe alım mülakatında “bu kodu ben yazdım” demek ne anlama gelir?

Bu sorulara Hafta 10 ve Hafta 12’de tekrar döneceğiz. Bugün amaç bir yanıt bulmak değil, soruyu görmek.

Yazılım geliřtirmenin adımları



- Bu adımlar her projede vardır; sıralaması ve tekrarı süreç seęimine göre deęiřir.
- Kesikli ok önemlidir: bakım sırasında yeni gereksinim doğar, döngü bařtan iřler.

Adımlar ve bu dersteki yeri

Adım	Yanıtladığı soru	Hafta
Fizibilite	Bu proje yapılmalı mı, yapılabilir mi?	3
Gereksinim	Müşteri ve kullanıcı ne bekliyor?	4, 5
Kullanıcı deneyimi	Kullanıcı bunu nasıl kullanacak?	7
Sistem tasarımı	Sistem hangi parçalardan oluşacak?	9
Program tasarımı ve kodlama	Kod nasıl örgütlenecek?	10
Test ve güvenilirlik	Doğru çalıştığını nasıl kanıtlarız?	11
Teslimat ve bakım	Müşteri nasıl kurar, kim bakar?	12

Dört ödeviniz bu adımlarla hizalıdır. Bakım çoğu sistemde en büyük maliyet kalemidir; “bitti” demek nadiren doğrudur.

“Test” kelimesi üç ayrı şeyi anlatır

- **Birim testi:** tek bir fonksiyon veya sınıf doğru mu? Geliştirici yazar.
- **Sistem testi:** parçalar birlikte doğru çalışıyor mu? Ekip yürütür.
- **Kabul testi:** müşterinin istediği iş yapılıyor mu? Müşteri yürütür.

Sık karışan nokta

“Test ettim, çalışıyor” cümlesi genelde “kendi bilgisayarımda bir kez denedim” anlamına gelir. Bu üçünden hiçbiri değildir.

Üçünü Hafta 11’de ayrıntılı, kabul testini Hafta 12’de müşteriyle birlikte ele alacağız.

**Uygulamalı örnek: kapsamı bir
yarıyla sığdırmak**

Grubun ilk fikri

“Kampüs için bir süper uygulama yapalım: yemek siparişi, servis takibi, ders programı, kulüp etkinlikleri, ikinci el pazarı, oda arkadaşı bulma ve bir de sohbet.”

- Bu fikir bir yarıyılıda bitmez; yedi ayrı ürün içerir.
- Her modül ayrı müşteri, ayrı mevzuat ve ayrı veri modeli demektir.
- Sonuç genellikle şudur: yedi modülün hiçbiri kullanılabilir düzeye gelmez.

Şimdi bu fikri adım adım daraltalım.

Uygulamalı örnek: daraltma adımları

▶ Adım 1 — Gerçek bir müşteri seç

Yedi modülün her birinin müşterisi farklı. Kantin işletmecisi somut, ulaşılabilir ve haftalık görüşmeye açık. **Karar:** yemek siparişi modülü.

▶ Adım 2 — Müşterinin en önemli sorununu bul

İşletmeci diyor ki: “Öğle arasında kuyruk 20 dakika; öğrenciler sıraya girmeden sipariş verebilse yeter.” **Karar:** önceden sipariş ve teslim saati.

▶ Adım 3 — Uçtan uca en ince dilimi tanımla

Menüyü gör, sipariş ver, teslim saati seç, işletmeci listeyi görsün. Ödeme ilk sürümde nakit; kart sonra.

Uygulamalı örnek: sonuç

Kapsam içi (bu yarıyıl)	Kapsam dışı (yazılı olarak)
Menü görüntüleme	Stok ve maliyet yönetimi
Sipariş verme ve teslim saati	Kurye ve adrese teslim
İşletmeci sipariş listesi ekranı	Muhasebe entegrasyonu
Basit kullanıcı girişi	Kampüs kimlik sistemi entegrasyonu
Sipariş durumu bildirimi	Çok dilli arayüz, kampanya motoru

- Kapsam dışı listesi, kapsam içi listesi kadar önemlidir; müşteriyle imzalanır.
- Daraltma, fıkri küçültmek değildir. Bitirilebilir bir ilk sürüm tanımlamaktır.
- Kalan modüller “gelecek sürüm” listesine yazılır, çöpe atılmaz.

Alıştırma (12 dk — grup çalışması)

Aşağıdaki fikri bir yarıyıl da bitirilebilir hâle getirin. Çıktı: tek cümlelik problem tanımı, müşteri, 5 maddelik kapsam içi ve 5 maddelik kapsam dışı listesi.

Ham fikir

“Üniversite kütüphanesi için akıllı bir sistem: kitap arama, çalışma odası rezervasyonu, kayıp eşya takibi, sessizlik ihlali bildirimi, kitap öneri motoru ve okuma grubu eşleştirme.”

Not: müşteri olarak kütüphane görevlisini varsayabilirsiniz. Gerçek projenizde bu varsayımı görüşmeyle doğrulamanız gerekecek.

Alıştırma 3: örnek çözüm

Çözüm

Problem: Çalışma odaları defterle rezerve ediliyor; aynı oda iki kişiye veriliyor ve boş odalar boş kalıyor.

Müşteri: Kütüphane hizmetleri sorumlusu.

Kapsam içi	Kapsam dışı
Oda ve saat listesini görmek	Kitap arama ve katalog
Rezervasyon yapmak ve iptal etmek	Öneri motoru
Çakışmayı engellemek	Kayıp eşya takibi
Görevli için doluluk ekranı	Sessizlik ihlali bildirimi
Gelmeyen rezervasyonu serbest bırakmak	Okuma grubu eşleştirme

- Tek bir temel soruna odaklanmak, altı modülü yarım bırakmaktan iyidir.
- “Çakışmayı engellemek” maddesi sistemin var olma nedenidir; ilk sürümde olmalıdır.

Bölüm 3: Dönem projesi

Neden gerçek bir proje?

- Yazılım mühendisliği, anlatılarak değil yapılarak öğrenilir.
- Gerçek müşteri, gereksinimlerin değişmesini ve çelişmesini deneyimler.
- Yedi ila on kişilik grup, eş güdüm sorununu gerçek kılar.
- Hatalarınızı burada yapın. Bedeli bir not; iş hayatında bir bütçe olur.

☰ Bu hafta projenizde

Dönem boyunca tek bir proje yürütürsünüz. Dört ödev, bu projenin dört kilometre taşıdır. Ayrı ayrı ödevler değil, aynı işin ardışık aşamalarıdır.

Kural 1: gerçek müşteri

- Müşteri, grup üyelerinden biri *olamaz*.
- Müşteri gerçek bir ihtiyacı olan bir kişi veya birim olmalıdır: bir akademik birim, bir kampüs işletmesi, bir yerel şirket, bir dernek, bir okul.
- Müşteri haftada en az bir kez, kısa da olsa görüşmeye açık olmalıdır.
- Müşteri projenin sonunda yazılımı kullanmaya niyetli olmalıdır.

En sık yapılan hata

“Tanıdığım bir esnaf var, olur der.” Görüşmeye zaman ayırmayan müşteri, müşteri değildir. İkinci bir aday bulundurun.

- Müşterisi olmayan, yalnızca “ilginç” bulunan teknoloji denemeleri.
- Hazır bir açık kaynak projeyi kurup arayüz rengini değiştirmek.
- Yalnızca bir ders ödevi veya sınav uygulaması olarak var olan sistemler.
- Yedi ayrı modülden oluşan ve hiçbiri bitmeyecek “süper uygulama” fikirleri.
- Gerçek kişisel veri toplayıp hukuki çerçevesi kurulmamış sistemler.

Fikrinizden emin değilseniz Hafta 2 dersinden önce bana yazın. Onay almak bir haftanızı kurtarır.

Kural 2: grup ve roller

Grup

- 7–10 kişi. Bu büyüklük organizasyon olmadan çalışamaz; amaç da budur.
- Herkes her alana katkı verir: görüşme, tasarım, kod, test, rapor, sunum.
- Uzman üye o alanı sürükler, ama tek başına yapmaz.
- Bireysel katkı commit, PR ve akran değerlendirmesinden izlenir.

Dönüşümlü roller (2 haftada bir)

- **Koordinatör:** görev dağıtır, panoyu güncel tutar.
- **İletişim sorumlusu:** müşteriyle tek temas noktası.
- **Not tutucu:** tutanağı yazar, depoya ekler.

Serbest yolcu

Katkı vermeyen üyeyi örtbas etmeyin; bedelini bütün grup öder. Erken haber verin.

Kural 3: haftalık çalışma düzeni

- **Müşteri görüşmesi:** haftada en az bir kez; tutanak 24 saat içinde depoya.
- **Grup içi toplantı:** haftada en az bir kez; kararlar yazılır.
- **Grup Dakikası:** her ders 1 slayt, 1 dakika.
- **Pano:** her iş bir kartta, her kartın bir sahibi.
- **Depo:** her hafta commit görülmeli; son haftada toplu yükleme kabul edilmez.

☰ Bu hafta projenizde

Bu düzen yalnızca not için değil, projeyi zamanında bitirebilmeniz için gereklidir. Bir hafta atlanması iki hafta kaybettirir.

14 haftalık takvim

Hafta	Konu	Proje adımı
1	Giriş, mesleki sorumluluk, dönem projesi	Grup kurma başlar
2	Modern süreçler; Git, GitHub, Kanban	Grup ve müşteri onayı
3	Fizibilite ve müşteri görüşmesi	Ödev 1 verilir
4	Gereksinim mühendisliği	İş birikimi oluşur
5	Gereksinim modelleri ve risk analizi	Ödev 1 teslim, Ödev 2 verilir
6	Hafif proje yönetimi	Sprint planı
7	Kullanıcı deneyimi ve prototipleme	Ödev 2 teslim
8	<i>Vize haftası, ders yok</i>	Görüşmeler sürer
9	Sistem mimarisi ve güvenlik	Ödev 3 verilir
10	Program tasarımı, desenler, yapay zekâ	Kodlama
11	Test, güvenilirlik, performans	Ödev 3 teslim, Ödev 4 verilir
12	Teslimat, hukuk, kariyer, sunum	Kabul testi
13-14	Proje sunumları	Ödev 4 teslim

Dört ödev, dört kilometre taşı

Ödev	İçerik	Verilme	Teslim
1	Fizibilite Raporu	Hafta 3	Hafta 5
2	Gereksinim ve Risk Raporu	Hafta 5	Hafta 7
3	Tasarım Raporu	Hafta 9	Hafta 11
4	Uygulama ve Final Raporu	Hafta 11	Hafta 14

- Her ödev bir öncekinin üstüne kurulur; ikinci ödev ilkini varsayar.
- Her ödevde bir katkı tablosu bulunur: kim hangi bölümü yazdı?
- Ayrıntılı yönergeler ve puanlama ölçütleri *odevler/* klasöründedir.

Müşteri onay formu

- Müşterinin adı, kurumu, iletişim bilgisi.
- Çözülecek problemin bir cümlelik tanımı.
- Haftalık görüşmeye katılım taahhüdü.
- İmza veya onay e-postası.

GitHub deposu

- Tüm üyeler ve öğretim üyesi ekli.
- *README*: proje, müşteri, kurulum.
- *docs/*: raporlar ve tutanaklar.
- Kanban panosu ve kilometre taşları.

İkisi de Hafta 2 dersinde teslim edilir. Şablonlar *sablonlar/* klasöründedir.

- Her grup çalışan ürününü canlı gösterir; slayt tek başına yeterli değildir.
- Müşteriniz varsa sunuma davet edin veya kabul görüşmesinin kaydını gösterin.
- Her üye konuşur. Tek kişinin sunduğu proje, grup projesi sayılmaz.
- Değerlendirme başlıkları: ürün, süreç, sunum, müşteri memnuniyeti.

Demo kuralı

Canlı demo başarısız olabilir. Yedek olarak 2 dakikalık ekran kaydı hazır bulundurun. Bu bir tedbir değil, mesleki alışkanlıktır.

Proje fikri nereden bulunur?

- Kendi kampüs hayatınızdaki aksaklıklar: kuyruklar, defterler, Excel dosyaları.
- Bölüm ve fakülte birimleri: laboratuvar rezervasyonu, staj takibi, arşiv.
- Kampüs işletmeleri: kantin, kırtasiye, fotokopi, spor salonu.
- Yakınınızdaki küçük işletmeler ve dernekler.
- “Bu iş neden hâlâ kâğıtla yapılıyor?” sorusu iyi bir başlangıçtır.

İyi bir işaret

Müşteri size süreci anlatırken bir dosya, defter veya karışık bir Excel çıkarıyorsa doğru yerdesiniz. Orada gerçek bir problem vardır.

Çıkış bileti: kendi cümlelerinizle

1. Rezervasyon projesinde müşteri ile son kullanıcıyı ayırın.
2. Çalışan kod dışında bir başarı ölçütü yazın.
3. Projenizde ilk doğrulayacağınız varsayım nedir?

Yanıt biçimi

Her soruya kısa bir yanıt yazın; kararınızı bir gerekçeyle destekleyin. Ardından yanınızdaki kişiyle karşılaştırın.

☰ Bu hafta projenizde

- Grubunuzu kurun (7–10 kişi) ve bir iletişim kanalı açın.
- En az 3 müşteri adayı belirleyin; her biri için “bu yazılımı neden kullanır?” sorusunu yanıtlayın.
- Adaylardan biriyle ön görüşme yapın: haftalık görüşmeye zaman ayırabilir mi?
- Grup listesini ve müşteri onay formunu Hafta 2 dersine getirin.
- GitHub hesabı olmayan üyeler hesap açsın.

- Sommerville, I. (2016). *Software Engineering*, 10. baskı, Pearson. Bölüm 1.
- Brooks, F. (1995). *The Mythical Man-Month*, genişletilmiş baskı. Addison-Wesley.
- CrowdStrike (2024). *Preliminary Post Incident Review*, Temmuz 2024.
- Post Office Horizon IT Inquiry. <https://www.postofficehorizoninquiry.org.uk>
- US House Committee (2020). *The Design, Development and Certification of the Boeing 737 MAX*.
- Kişisel Verileri Koruma Kurumu rehberleri. <https://www.kvkk.gov.tr>
- GitHub Docs. *Responsible use of GitHub Copilot*.
- Erciyes Üniversitesi eğitim-öğretim yönetmeliği, akademik dürüstlük maddeleri.

Gelecek hafta

Modern yazılım geliştirme süreçleri; Git, GitHub ve Kanban kurulumu.

Grup listenizi ve müşteri adaylarınızı getirin.